

Semester Project Proposal

CI/CD Pipeline Failure Prediction

Using Machine Learning on TravisTorrent Dataset

Project Proposal

Submitted by

Muhammad Bilal - BSCS23146

Abdul Basit - BSCS23023

Asharib - BSCS23038

Department of Computer Science
Information Technology University

Date: April 4th, 2026

1. Introduction and Background

The modern software development ecosystem relies heavily on **Continuous Integration and Continuous Deployment (CI/CD)** pipelines to automate the building, testing, and deployment of code. Platforms such as **Travis CI**, **GitHub Actions**, **GitLab CI**, and **Jenkins** are used by hundreds of thousands of teams worldwide to validate code changes before they reach production. Despite their widespread adoption, a significant proportion of these pipeline builds fail, resulting in wasted computational resources, delayed releases, and increased developer frustration.

Historically, developers have had no mechanism to anticipate build failures before triggering a pipeline run. A developer pushing a large or risky commit must wait for the entire build to complete, sometimes **30 to 60 minutes only to discover that the build has failed due to a missing dependency, a broken test, or a configuration error**. This reactive approach is costly in both time and resources, particularly in large organizations running thousands of builds per day.

This project proposes a predictive approach: rather than discovering failures after the fact, we aim to build a Machine Learning model that analyzes pre-build signals, such as commit size, code churn, programming language, time of push, contributor history, and repository activity and predicts whether a given pipeline build will succeed or fail before it is executed. This allows developers to receive early warnings and correct potential problems proactively.

The project targets the software engineering and DevOps domain. Development teams in technology companies, open-source organizations, and enterprise environments stand to benefit directly from this system. A reliable failure predictor reduces wasted CI/CD compute time, improves developer productivity, shortens feedback cycles, and enables intelligent build prioritization and resource allocation. The system has clear commercial value in any organization running automated pipelines at scale.

2. Problem Description and Project Objective

2.1 Problem Description

CI/CD pipeline failures are a well-documented source of inefficiency in software development. In large-scale open-source and commercial projects, build failure rates typically range from 20% to 40% of all triggered builds. Each failed build consumes server-side compute resources, occupies shared build queues, and blocks downstream developers waiting for test results or deployments. When failures occur repeatedly or in rapid succession, they erode developer confidence in the pipeline and lead to pipeline congestion.

The fundamental challenge is the absence of intelligent, pre-emptive analysis. Current CI/CD platforms execute every triggered build blindly, regardless of how risky or unstable the triggering commit appears. There is no mechanism for the system to say: 'this commit has a high probability of failing, alert the developer before consuming resources.' A Machine Learning model trained on historical build data can fill this gap by learning the patterns that distinguish successful builds from failed ones.

Key challenges in solving this problem include: handling class imbalance (successful builds outnumber failures), avoiding data leakage by using only features available before the build runs, engineering meaningful features from raw commit and repository metadata, and deploying the model in a form that is accessible and interpretable to non-technical stakeholders.

2.2 Project Objective

To design, train, and deploy a supervised Machine Learning system that predicts the success or failure of a CI/CD pipeline build prior to its execution, using historical build metadata from the TravisTorrent dataset, with the goal of reducing wasted computational resources and enabling proactive developer intervention.

2.3 Success Metrics

Metric	Target and Rationale
Recall (Primary)	Target > 80% (maximize detection of actual build failures)
F1 Score	Target > 0.78 (balanced measure on imbalanced classes)
ROC-AUC	Target > 0.85 (strong class separation ability)
Precision	Target > 0.75 (minimize false failure alarms)
Inference Latency	Prediction must complete in under 200ms for real-time usability

3. System Architecture

The proposed system follows a modular ML pipeline architecture with four primary components: a data layer, a model layer, a prediction API layer, and a user interface layer. Each component is independently developed and communicates through standardized file interfaces and function calls.

Component	Description
Data Layer	TravisTorrent CSV → Pandas pipeline → Cleaned and feature-engineered dataset saved to data/processed/

Model Layer	Scikit-learn / XGBoost / LightGBM training pipeline → Best model serialized via joblib to models/best_model.pkl
Prediction Layer	predict.py function — accepts raw build metadata dict → loads model → returns failure probability (0.0–1.0) and binary label
UI Layer	Streamlit web dashboard — input form → calls predict.py → displays result with probability, gauge chart, and SHAP-based failure reasons

3.1 Data Flow Diagram

The end-to-end data flow is as follows:

[TravisTorrent Raw CSV] → [Data Cleaning & Validation] → [Feature Engineering] → [Encoding & Scaling] → [Train / Val / Test Split] → [Model Training] → [Hyperparameter Tuning] → [Best Model (PKL)] → [Prediction Pipeline] → [Web Dashboard]

4. Methodology and Scope

4.1 Technical Approach

The project will follow a structured, reproducible ML pipeline consisting of the following sub-tasks executed in order:

1. Exploratory Data Analysis (EDA): Visualize class distribution, failure rates by language, failure by time of day and day of week, correlation heatmap, and outlier analysis using Matplotlib and Seaborn.
2. Data Preprocessing: Handle missing values in test result columns (fill with 0 where tests were not run). Encode target variable: tr_status passed → 0, broken/errored → 1, canceled → drop. Remove outlier builds (duration = 0 seconds or > 24 hours). Drop identifier columns (tr_build_id) that cannot contribute to generalization.
3. Feature Engineering: Derive time-based features (hour_of_day, day_of_week, is_weekend), commit-based features (total_code_churn = src_churn + test_churn, test_to_src_ratio, is_large_commit flag), and repository-level features (author_failure_rate, recent_failure_rate as rolling averages).
4. Encoding and Scaling: Label-encode gh_lang (programming language). Apply Standard Scaler to numeric features. Convert boolean columns to integer. Fit scaler only on training set to prevent data leakage.
5. Dataset Splitting: 70% training, 15% validation, 15% held-out test set.
6. Class Imbalance Handling: Apply SMOTE on training set only. Additionally configure class_weight='balanced' on applicable models.

7. Model Training: Train five models — Logistic Regression (baseline), Random Forest, Gradient Boosting, XGBoost, and LightGBM.
8. Hyperparameter Tuning: Apply RandomizedSearchCV with 5-fold cross-validation on the top two performing models.
9. Model Evaluation: Evaluate all models on test set using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Generate confusion matrices, ROC curves, and feature importance plots.
10. Model Interpretation: Apply SHAP (SHapley Additive exPlanations) on the best model to generate global and per-prediction explanations.
11. Model Serialization: Save final model and scaler using joblib. Build predict.py prediction function.
12. Dashboard Development: Build a five-page Streamlit web application with live prediction, probability display, SHAP-based failure reasons, EDA charts, and model performance visualizations.

4.2 Baseline and Advanced Models

Model	Role and Justification
Logistic Regression	BASELINE (Simple linear model. Establishes minimum acceptable performance. Fast to train, highly interpretable via coefficients. Requires scaled features.)
Random Forest	INTERMEDIATE (Ensemble of decision trees. Robust to outliers, handles raw features, provides feature importance. Good baseline for tree-based methods.)
Gradient Boosting	ADVANCED (Sequential boosting. Generally outperforms Random Forest at cost of longer training time. Strong regularization.)
XGBoost	ADVANCED (Optimized gradient boosting with L1/L2 regularization, native missing value handling, and high computational efficiency.)
LightGBM	ADVANCED (Histogram-based gradient boosting. Fastest training on large datasets. Typically achieves highest accuracy on tabular tasks.)

The Logistic Regression model serves as the mandatory baseline for performance benchmarking. If a more complex model (e.g., XGBoost) does not significantly outperform Logistic Regression on F1 and ROC-AUC, this indicates an issue in feature engineering rather than model complexity, enabling early-stage debugging.

4.3 Preprocessing Steps Summary

- Missing value imputation: zero-fill for test result columns (tr_tests_ok, tr_tests_fail, tr_tests_run)

- Target encoding: binary label from tr_status, drop canceled rows
- Outlier removal: cap tr_duration at 1 second minimum and 24 hours maximum
- Feature normalization: Standard Scaler applied to all continuous numeric features
- Categorical encoding: Label encoding for programming language (gh_lang)
- Boolean conversion: gh_is_pr, is_weekend converted to integer 0/1
- Class imbalance: SMOTE applied on training set only after the train/val/test split

4.4 Project Scope

In Scope:

- Data cleaning, feature engineering, and preprocessing of TravisTorrent dataset
- Training and evaluation of five supervised ML models
- Hyperparameter tuning and cross-validation
- Model interpretation using SHAP values
- Streamlit web dashboard with live prediction interface
- Model serialization and a reusable prediction pipeline function

Out of Scope:

- Real-time integration with any live Travis CI or GitHub Actions API
- Processing raw build log text (NLP on error messages is not included)
- Time-series forecasting of future build trends across projects
- Deployment to a cloud server or containerization with Docker/Kubernetes
- Support for programming languages outside those present in the TravisTorrent dataset

4.5 AI Ethics Considerations

This project raises several ethical considerations that must be acknowledged:

- **Dataset Bias:** The TravisTorrent dataset is restricted to Ruby and Java projects on GitHub, which introduces language bias. Models trained on this data may not generalize reliably to other languages such as Python or C++. All predictions should be interpreted with this limitation in mind.
- **Developer Profiling:** Features such as author_failure_rate use historical contributor behavior to make predictions. This risks penalizing developers with less experience or those working on more experimental branches. The model should not be used as a performance evaluation tool for individual developers.
- **Data Privacy:** TravisTorrent uses public GitHub repository data. No personally identifiable information is processed beyond public GitHub usernames. The project complies with open-source data sharing norms.

- **Misuse Risk:** If deployed in a production CI/CD environment, a high false positive rate (predicting failure when the build would pass) could block legitimate developer work. Threshold calibration must prioritize recall while keeping false positive rate manageable.

5. Dataset

5.1 Dataset Details

Attribute	Details
Dataset Name	TravisTorrent
Source / Link	https://travistorrent.testroots.org Also archived at: https://figshare.com/articles/dataset/TravisTorrent/19314170
Published By	Moritz Beller, Georgios Gousios, Andy Zaidman — TU Delft (IEEE/ACM MSR 2017)
Total Records	2,640,825 build records (we will use a filtered subset of ~300,000 to 500,000 builds)
Total Features	55 features per build row (we will use ~20 after cleaning and dropping irrelevant columns)
Dataset Type	Tabular / Structured CSV data
Languages Covered	Ruby (936 projects) and Java (423 projects) — 1,359 total GitHub projects
Data Sources	Three merged sources: Travis CI API (tr_prefix), GitHub repository (git_prefix), GHTorrent (gh_prefix)
Target Variable	tr_status — values: passed (0), broken (1), errored (1), canceled (dropped)
Class Distribution	Approximately 70% passed builds and 30% failed builds — moderate class imbalance
File Format	CSV (1.8 GB uncompressed, ~200 MB compressed)
License	Publicly available research dataset — free for academic use

5.2 Key Features Used

Feature Name	Description
tr_status	Target — passed / broken / errored / canceled
tr_duration	Total build duration in seconds
tr_tests_run	Total number of test cases executed

tr_tests_fail	Number of test cases that failed
gh_lang	Programming language of the project
gh_team_size	Number of contributors in the repository
gh_num_commits_in_push	Number of commits included in the triggering push
gh_is_pr	Whether the build was triggered by a Pull Request
git_diff_src_churn	Lines of production source code changed
git_diff_test_churn	Lines of test code changed
git_num_commits	Number of commits in the push event

6. Tools and Technology

Tool / Technology	Purpose and Justification
Operating System	Windows 10/11 or Ubuntu 22.04 LTS — any platform supporting Python 3.10+
Programming Language	Python 3.10 — industry standard for ML development
Data Manipulation	Pandas 2.x — DataFrame operations, cleaning, feature engineering
Numerical Computing	NumPy — array operations and mathematical transformations
Machine Learning	Scikit-learn — preprocessing, Logistic Regression, Random Forest, Gradient Boosting, evaluation metrics
Boosting Models	XGBoost and LightGBM — advanced gradient boosting classifiers
Class Imbalance	Imbalanced-learn (SMOTE) — synthetic oversampling of minority class
Model Interpretation	SHAP — SHapley Additive exPlanations for global and local model explanations
Visualization	Matplotlib and Seaborn — EDA charts, confusion matrices, ROC curves
Model Serialization	Joblib — save and load trained models and scalers
Web Dashboard	Streamlit — Python-native interactive web application framework
Interactive Charts	Plotly — interactive visualizations inside the dashboard
Development Env	Jupyter Notebook — exploratory analysis and documentation
Version Control	Git + GitHub — shared repository for collaboration
Hardware	Minimum 8 GB RAM. GPU not required for this dataset size.

7. Contributions

The project workload is distributed across three team members based on the natural phases of the ML pipeline. Each member owns a specific set of deliverables and is accountable for their quality.

Name	Enrolment No.	Contribution	Role
Muhammad Bilal	BSCS23146	Data cleaning and preprocessing. Missing value handling. Target variable encoding. Exploratory Data Analysis (EDA) notebook. All EDA visualizations. Feature engineering. time, commit, and repository features. Encoding and scaling. Processed dataset preparation.	Data Engineer
Asharib	BSCS23038	Model training for all five models. Class imbalance handling (SMOTE). Model evaluation on validation and test sets. Hyperparameter tuning (RandomizedSearchCV + 5-fold CV). SHAP value computation and interpretation. Evaluation visualizations — confusion matrix, ROC curve, feature importance. Model serialization (PKL). Prediction pipeline (predict.py).	ML Engineer
Abdul Basit	BSCS23023	Streamlit web dashboard (all five pages). Prediction form connected to model. Probability display, gauge chart, SHAP-based reasons. EDA chart embedding. Model performance page. README.md and requirements.txt documentation. Final integration testing.	Web Developer & Integration Lead

References

- [1] M. Beller, G. Gousios, and A. Zaidman, "TravisTorrent: Synthesizing Travis CI and GitHub for Full-Stack Research on Continuous Integration," in Proceedings of the 14th International Conference on Mining Software Repositories (MSR), IEEE/ACM, 2017.
- [2] E. Rzig, F. Hassan, and H. Malik, "Characterizing and Predicting Build Failures in Continuous Integration," in Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2020.
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
- [5] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [6] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems (NeurIPS), 2017. — (SHAP library)
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.
- [8] TravisTorrent Dataset Archive, Figshare, 2022. [Online]. Available: <https://figshare.com/articles/dataset/TravisTorrent/19314170>